Reporte

| | |
|---|---|
| **Integrantes**:<br>Vanessa Judith Garza Fernández 2145305<br>Andrick Fabian Moreno Torres 2147624<br>Edmundo Santos Ramírez 2148132 | |
| **Nombre del curso:** Componentes de lógica de negocio | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
| **Modulo Netacad**: 3 | **Actividad**: ADRs |
| **Fecha**: 15/10/2026 | |

# FARMAEXPRESS - ARCHITECTURE DECISION RECORD

## ADR-01: Selección del Patrón de Lógica de Negocio (Domain Model)

- **Título:** Elección del patrón Domain Model para la gestión de ventas, recetas y control de lotes.

- **Estado:** Aceptada.

- **Contexto:**

  FarmaExpress opera múltiples sucursales con control estricto de inventario por producto, lote y fecha de vencimiento. La operación exige aplicar invariantes y reglas de negocio críticas de forma simultánea: asignación y despacho por política FEFO (*First Expired, First Out*), validación y retención obligatoria de recetas para medicamentos controlados, bloqueo inmediato de lotes en cuarentena sanitaria y cálculo de puntos de reorden. Implementar estas reglas en scripts procedimentales generaría duplicación de validaciones, riesgo de dispensar medicamento caducado y dificultad para auditar el cumplimiento normativo.

- **Alternativas:**

    1. *Transaction Script:* Procedimientos lineales independientes por pantalla o caso de uso. Ventaja: rápido de codificar en flujos iniciales. Consecuencia/Riesgo: fragmenta y duplica la lógica de negocio (como la verificación de caducidad y retención de recetas) en múltiples lugares, haciéndola frágil ante cambios.

    2. *Table Module:* Estructuración de la lógica centrada en tablas de la base de datos. Ventaja: integración simple con esquemas relacionales directos. Consecuencia/Riesgo: no encapsula adecuadamente relaciones de comportamiento ricas entre entidades (ej. asociar una Venta con una RecetaMedica y reservar existencias de un Lote específico).

    3. *Domain Model:* Modelo orientado a objetos con entidades ricas, Value Objects, Aggregates e invariantes de negocio autocontenidas.

- **Decisión:**

    Se adopta **Domain Model** como patrón principal de la capa de dominio. Toda la lógica de validación sanitaria, control de fechas de vencimiento, retención de recetas y consistencia de existencias residirá dentro de los Aggregates (Venta, InventarioLote) y sus Value Objects (RecetaMedica, FechaCaducidad).

- **Consecuencias:**

    - *Beneficios:* Centralización y protección estricta de las invariantes sanitarias, alta reusabilidad, pruebas unitarias del dominio sin depender de base de datos y desacoplamiento de la infraestructura.

    - *Costos / Riesgos:* Mayor complejidad técnica inicial, requiere el diseño de mapeadores y abstracción de repositorios hacia la capa de persistencia.

- **Trazabilidad:**

    RN-01 (Política FEFO), RN-02 (Bloqueo por caducidad), RN-03 (Medicamento controlado), US-01, US-02, CU-01, CU-02.

# ADR-02: Delimitación Transaccional con Repository y Unit of Work

- **Título:** Aislamiento de persistencia mediante Repository y delimitación transaccional con Unit of Work.

- **Estado:** Aceptada.

- **Contexto:**

  Durante una venta en mostrador se deben ejecutar operaciones dependientes de manera atómica: descontar el stock del lote asignado bajo FEFO, asociar y marcar la receta como retenida, y generar el registro de la venta con su ticket fiscal. Si alguna operación falla (por ejemplo, falta de stock concurrente o error al asentar la receta), el sistema no debe permitir inconsistencias entre el inventario físico y el inventario registrado.

- **Alternativas:**

  1. *Acceso Directo con Sentencias SQL / Transacciones en Controladores:* Gestionar aperturas, transacciones y conexiones directamente en la capa de presentación o servicios. Consecuencia/Riesgo: alto acoplamiento a la base de datos, fugas de conexiones y dificultad de mantenimiento.

  2. *Active Record:* Entidades acopladas a la persistencia con métodos directos de guardado (venta.save()). Consecuencia/Riesgo: viola el principio de responsabilidad única (SRP) y expone detalles de persistencia en el modelo de dominio.

  3. *Repository con Unit of Work:* Interfaces de repositorio que representan colecciones del dominio en memoria (IVentaRepository, IInventarioLoteRepository) y un coordinador transaccional (IUnitOfWork) que gestiona el commit o rollback integral.

- **Decisión:**

  Se implementa el patrón **Repository** para desacoplar el dominio de los detalles de base de datos y **Unit of Work** para asegurar que todos los cambios sobre

los Aggregates de una misma operación se confirmen o cancelen bajo una única transacción ACID.

- **Consecuencias:**

  - *Beneficios:* Garantiza consistencia e integridad transaccional, elimina discrepancias entre saldo físico y saldo en sistema, y facilita el cambio de motor de base de datos sin tocar la capa de dominio.

  - *Costos / Riesgos:* Incrementa el número de clases e interfaces a implementar y mantener en la capa de infraestructura.

- **Trazabilidad:**

  RN-05 (Ajustes y control de inventario), US-10, US-11, CU-01, DS-01 (Diagrama de secuencia de venta).

# ADR-03: Separación de Consultas Multisucursal y Comandos de Venta (CQRS Básico)

- **Título:** Adopción de CQRS básico para segregar comandos de venta de consultas de existencias inter-sucursal.

- **Estado:** Aceptada.

- **Contexto:**

  FarmaExpress presenta un problema crítico de visibilidad entre sucursales. Los cajeros requieren consultar existencias en otras tiendas en tiempo real y buscar sustitutos bioequivalentes ante desabastos sin degradar el rendimiento del punto de venta en mostrador ni bloquear las transacciones transaccionales de cobro.

- **Alternativas:**

  1. *Modelo CRUD Unificado:* Utilizar las mismas entidades de dominio complejas tanto para transaccionar ventas como para responder búsquedas y listados en pantalla. Consecuencia/Riesgo: consultas pesadas, sobrecarga de memoria innecesaria y posibles bloqueos de concurrencia sobre los registros de inventario.

  2. *CQRS Completo con Event Sourcing y Bases de Datos Separadas:* Duplicar bases de datos físicas independientes (escritura relacional y lectura NoSQL) sincronizadas de forma asíncrona. Consecuencia/Riesgo: costo excesivo de infraestructura y complejidad de consistencia eventual no justificada para el alcance del sistema.

  3. *CQRS Básico a Nivel de Aplicación:* Mantener la misma base de datos relacional pero separar a nivel de código los *Commands* (operaciones que mutan estado usando Domain Model) de las *Queries* (consultas optimizadas que devuelven DTOs planos directamente a la interfaz).

- **Decisión:**

  Adoptar **CQRS básico a nivel de aplicación**. Las operaciones de mutación de estado (RegistrarVentaCommand, BloquearLoteCommand) serán coordinadas por Application Services usando el Domain Model, mientras que las consultas de consulta y lectura (ConsultarStockMultisucursalQuery, BuscarSustitutoGenericoQuery) ejecutarán proyecciones ligeras que retornan View Models sin instanciar Aggregates completos.

- **Consecuencias:**

  - *Beneficios:* Tiempos de respuesta óptimos en caja para consultas multisucursal, interfaces de lectura desacopladas y protección de los modelos de escritura.

  - *Costos / Riesgos:* Duplicidad intencional en la definición de modelos (DTOs de lectura separados de las entidades del dominio).

- **Trazabilidad:**

  US-04 (Consulta multisucursal), US-08 (Sugerencia de sustitutos), Etapa 6 de la guía de Unidad 3.